МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ

УЧРЕЖДЕНИЕ ОБРАЗОВАНИЯ

«БРЕСТСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»

ФАКУЛЬТЕТ ЭЛЕКТРОННО-ИНФОРМАЦИОННЫХ СИСТЕМ

Кафедра интеллектуальных информационных технологий

Отчет по лабораторной работе №7

Специальность ПО-12

Выполнил

С. Э. Михновец,

студент группы ПО-12

Проверила

А. Д. Кулик,

преп. стаж. кафедры ИИТ,

«___k _____2026 г.

Брест 2026

1. Изложение цели работы

Цель работы: освоить возможности языка программирования Python в разработке оконных графических приложений, изучить принципы объектно-ориентированного программирования при создании графических примитивов, а также реализовать алгоритмы построения фракталов и обработки событий таймера для динамических сцен.

2. Задание по лабораторной работе (Вариант 1 (15))

Задание 1. Построение графических примитивов и надписей.

Создать класс Triangle и класс Point.

Объявить список из

объектов класса Point.

Написать функцию, определяющую, какая из точек лежит внутри, а какая – снаружи треугольника.

Реализовать динамическую визуализацию с возможностью приостановки, изменения параметров скорости «на лету» и снятия скриншотов.

Задание 2. Реализация фрактала.

Построить множество Мандельброта.

Предусмотреть ввод параметров, влияющих на внешний вид фрактала (масштаб, количество итераций, смещение).

3. Спецификации ввода-вывода программы

Входные данные (вводятся через графический интерфейс):

Для Задания 1: Координаты вершин треугольника (заданы по умолчанию, но могут быть изменены в коде), количество генерируемых точек

(целое число), скорость анимации (в миллисекундах, целое число). Команды управления: «Старт», «Пауза/Продолжить», «Скриншот».

Для Задания 2: Максимальное количество итераций (целое число), масштаб (вещественное число), смещение по осям X и Y (вещественные числа).

Выходные данные:

Графическое окно приложения, разделенное на вкладки (Task 1 и Task 2).

Динамическая отрисовка точек (зеленые — внутри треугольника, красные — снаружи).

Графическое представление множества Мандельброта.

Файлы изображений (скриншоты) формата .png, сохраняемые в текущую директорию.

4. Текст программы

Программа использует встроенную библиотеку tkinter для создания GUI и PIL (Pillow) для генерации фрактала и создания скриншотов.

```
"""
Лабораторная работа №7.
Построение графических примитивов и фракталов с использованием Tkinter.
"""

import os
import random
import time
import tkinter as tk
from tkinter import ttk, messagebox

from PIL import Image, ImageTk, ImageGrab


# pylint: disable=too-few-public-methods
class Point:
    """Класс, описывающий точку на плоскости."""

    def __init__(self, x, y):
        self.x = x
        self.y = y


# pylint: disable=too-few-public-methods
class Triangle:
    """Класс, описывающий треугольник."""

    def __init__(self, p1, p2, p3):
        self.p1 = p1
        self.p2 = p2
        self.p3 = p3
```

```
    def is_point_inside(self, pt):
        """Определяет, находится ли точка внутри треугольника (векторное
произведение)."""
        def sign(point1, point2, point3):
            return (point1.x - point3.x) * (point2.y - point3.y) - \
                   (point2.x - point3.x) * (point1.y - point3.y)

        d1 = sign(pt, self.p1, self.p2)
        d2 = sign(pt, self.p2, self.p3)
        d3 = sign(pt, self.p3, self.p1)

        has_neg = (d1 < 0) or (d2 < 0) or (d3 < 0)
        has_pos = (d1 > 0) or (d2 > 0) or (d3 > 0)

        return not (has_neg and has_pos)


# pylint: disable=too-many-instance-attributes
class App:
    """Главный класс графического приложения."""

    def __init__(self, root_window):
        self.root = root_window
        self.root.title("Лабораторная работа №7")
        self.root.geometry("800x600")

        self.is_paused = False
        self.is_running = False
        self.points_to_draw = []
        self.triangle = None
        self.fractal_img_tk = None

        self.entry_n = None
        self.entry_speed = None
        self.btn_pause = None
        self.canvas1 = None

        self.entry_iter = None
        self.entry_zoom = None
        self.entry_ox = None
        self.entry_oy = None
        self.canvas2 = None

        self.notebook = ttk.Notebook(self.root)
        self.notebook.pack(fill='both', expand=True)

        self.tab1 = ttk.Frame(self.notebook)
        self.tab2 = ttk.Frame(self.notebook)

        self.notebook.add(self.tab1, text="Задание 1: Треугольник и
точки")
        self.notebook.add(self.tab2, text="Задание 2: Фрактал
Мандельброта")

        self.setup_tab1()
        self.setup_tab2()

    def setup_tab1(self):
        """Создает элементы управления для первой вкладки."""
        control_frame = ttk.Frame(self.tab1)
        control_frame.pack(side=tk.TOP, fill=tk.X, padx=5, pady=5)

        ttk.Label(control_frame, text="Кол-во точек
(n):").pack(side=tk.LEFT)
```

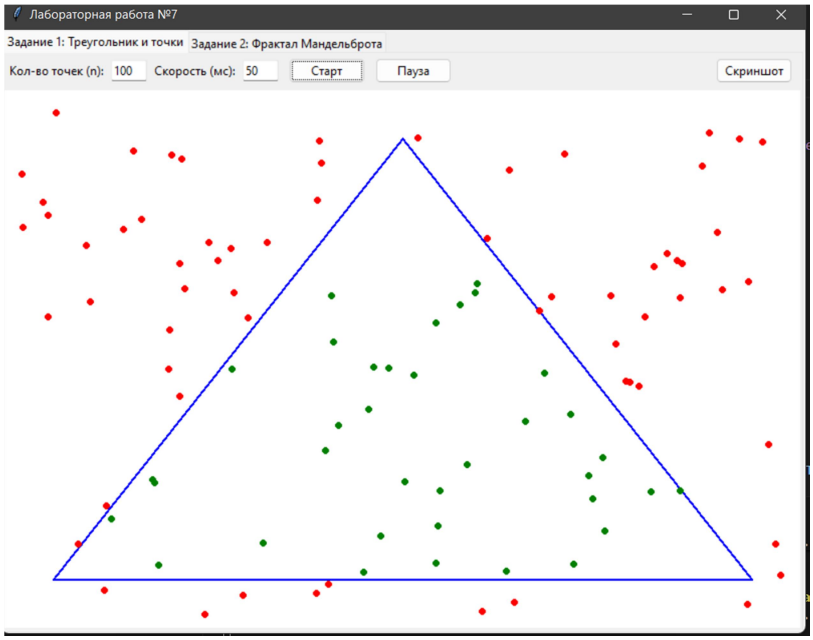
```
        self.entry_n = ttk.Entry(control_frame, width=5)
        self.entry_n.insert(0, "100")
        self.entry_n.pack(side=tk.LEFT, padx=5)

        ttk.Label(control_frame, text="Скорость (мс):").pack(side=tk.LEFT)
        self.entry_speed = ttk.Entry(control_frame, width=5)
        self.entry_speed.insert(0, "50")
        self.entry_speed.pack(side=tk.LEFT, padx=5)

        btn_start = ttk.Button(control_frame, text="Старт",
command=self.start_animation)
        btn_start.pack(side=tk.LEFT, padx=5)

        self.btn_pause = ttk.Button(control_frame, text="Пауза",
command=self.toggle_pause)
        self.btn_pause.pack(side=tk.LEFT, padx=5)

        btn_screen = ttk.Button(
            control_frame,
            text="Скриншот",
            command=lambda: self.take_screenshot(self.canvas1)
        )
        btn_screen.pack(side=tk.RIGHT, padx=5)

        self.canvas1 = tk.Canvas(self.tab1, bg="white")
        self.canvas1.pack(fill=tk.BOTH, expand=True)

    def start_animation(self):
        """Запускает анимацию отрисовки точек."""
        self.canvas1.delete("all")
        self.is_paused = False
        self.is_running = True

        try:
            n = int(self.entry_n.get())
        except ValueError:
            messagebox.showerror("Ошибка", "Введите целое число для n")
            return

        w = self.canvas1.winfo_width()
        h = self.canvas1.winfo_height()
        if w <= 1:
            w, h = 800, 500

        p1 = Point(w // 2, 50)
        p2 = Point(50, h - 50)
        p3 = Point(w - 50, h - 50)
        self.triangle = Triangle(p1, p2, p3)

        self.canvas1.create_polygon(
            p1.x, p1.y, p2.x, p2.y, p3.x, p3.y,
            outline='blue', fill='', width=2
        )

        self.points_to_draw = [
            Point(random.randint(10, w - 10), random.randint(10, h - 10))
            for _ in range(n)
        ]
        self.draw_next_point()

    def draw_next_point(self):
        """Отрисовывает следующую точку из списка с учетом таймера."""
        if not self.is_running or not self.points_to_draw:
            return
```

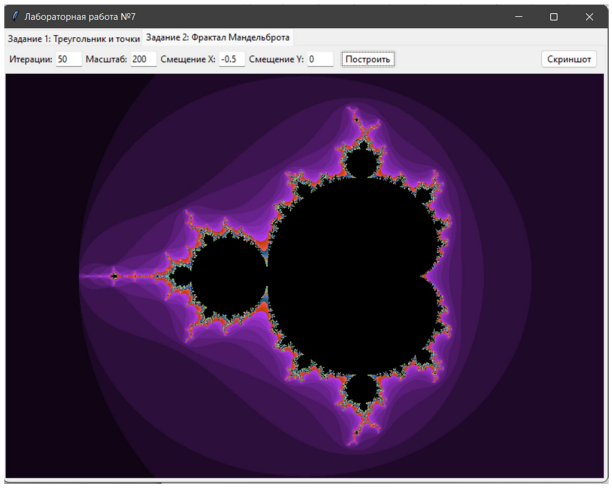
```
        if not self.is_paused:
            pt = self.points_to_draw.pop(0)

            if self.triangle.is_point_inside(pt):
                color = "green"
            else:
                color = "red"

            r = 3
            self.canvas1.create_oval(
                pt.x - r, pt.y - r, pt.x + r, pt.y + r,
                fill=color, outline=color
            )

        try:
            speed = int(self.entry_speed.get())
        except ValueError:
            speed = 50

        self.root.after(speed, self.draw_next_point)

    def toggle_pause(self):
        """Переключает состояние паузы анимации."""
        self.is_paused = not self.is_paused
        self.btn_pause.config(text="Продолжить" if self.is_paused else
"Пауза")

    def setup_tab2(self):
        """Создает элементы управления для второй вкладки."""
        control_frame = ttk.Frame(self.tab2)
        control_frame.pack(side=tk.TOP, fill=tk.X, padx=5, pady=5)

        ttk.Label(control_frame, text="Итерации:").pack(side=tk.LEFT)
        self.entry_iter = ttk.Entry(control_frame, width=5)
        self.entry_iter.insert(0, "50")
        self.entry_iter.pack(side=tk.LEFT, padx=2)

        ttk.Label(control_frame, text="Масштаб:").pack(side=tk.LEFT)
        self.entry_zoom = ttk.Entry(control_frame, width=5)
        self.entry_zoom.insert(0, "200")
        self.entry_zoom.pack(side=tk.LEFT, padx=2)

        ttk.Label(control_frame, text="Смещение X:").pack(side=tk.LEFT)
        self.entry_ox = ttk.Entry(control_frame, width=5)
        self.entry_ox.insert(0, "-0.5")
        self.entry_ox.pack(side=tk.LEFT, padx=2)

        ttk.Label(control_frame, text="Смещение Y:").pack(side=tk.LEFT)
        self.entry_oy = ttk.Entry(control_frame, width=5)
        self.entry_oy.insert(0, "0")
        self.entry_oy.pack(side=tk.LEFT, padx=2)

        btn_draw_frac = ttk.Button(
            control_frame,
            text="Построить",
            command=self.draw_fractal
        )
        btn_draw_frac.pack(side=tk.LEFT, padx=5)

        btn_screen2 = ttk.Button(
            control_frame,
            text="Скриншот",
            command=lambda: self.take_screenshot(self.canvas2)
```

```
        )
        btn_screen2.pack(side=tk.RIGHT, padx=5)

        self.canvas2 = tk.Canvas(self.tab2, bg="black")
        self.canvas2.pack(fill=tk.BOTH, expand=True)

    # pylint: disable=too-many-locals
    def draw_fractal(self):
        """Генерирует и отрисовывает фрактал Мандельброта."""
        w = self.canvas2.winfo_width()
        h = self.canvas2.winfo_height()
        if w < 10:
            w, h = 800, 500

        try:
            max_iter = int(self.entry_iter.get())
            zoom = float(self.entry_zoom.get())
            offset_x = float(self.entry_ox.get())
            offset_y = float(self.entry_oy.get())
        except ValueError:
            messagebox.showerror("Ошибка", "Некорректные параметры
фрактала")
            return

        img = Image.new('RGB', (w, h), 'black')
        pixels = img.load()

        for x in range(w):
            for y in range(h):
                cx = (x - w / 2) / zoom + offset_x
                cy = (y - h / 2) / zoom + offset_y
                c = complex(cx, cy)
                z = 0j
                i = 0
                for i in range(max_iter):
                    if abs(z) > 2.0:
                        break
                    z = z * z + c

                if i < max_iter - 1:
                    r = (i * 15) % 256
                    g = (i * 5) % 256
                    b = (i * 20) % 256
                    pixels[x, y] = (r, g, b)

        self.fractal_img_tk = ImageTk.PhotoImage(img)
        self.canvas2.create_image(0, 0, anchor=tk.NW,
image=self.fractal_img_tk)

    # pylint: disable=broad-exception-caught
    def take_screenshot(self, canvas):
        """Сохраняет скриншот выбранного холста."""
        self.root.update()
        x = canvas.winfo_rootx()
        y = canvas.winfo_rooty()
        w = canvas.winfo_width()
        h = canvas.winfo_height()

        try:
            img = ImageGrab.grab(bbox=(x, y, x + w, y + h))
            filename = f"screenshot_{int(time.time())}.png"
            img.save(filename)
            msg = f"Скриншот сохранен в:\n{os.path.abspath(filename)}"
            messagebox.showinfo("Успех", msg)
```

```
        except Exception as e:
            messagebox.showerror("Ошибка", f"Не удалось сделать
скриншот:\n{str(e)}")


if __name__ == "__main__":
    main_window = tk.Tk()
    app = App(main_window)
    main_window.mainloop()
```

Рисунки с результатами работы программы





Вывод: В ходе выполнения лабораторной работы были освоены принципы создания графических приложений на языке Python с использованием библиотеки tkinter.